compared to the system time zone, which `Date.today` uses by default. This means `Date.today` may equal `Date.yesterday`.

15.1.2 Named dates

15.1.2.1 `prev_year`, `next_year`

In Ruby 1.9 `prev_year` and `next_year` return a date with the same day/month in the last or next year:

```
d = Date.new(2010, 5, 8) # => Sat, 08 May 2010
d.prev_year              # => Fri, 08 May 2009
d.next_year              # => Sun, 08 May 2011
```

If date is the 29th of February of a leap year, you obtain the 28th:

```
d = Date.new(2000, 2, 29) # => Tue, 29 Feb 2000
d.prev_year               # => Sun, 28 Feb 1999
d.next_year               # => Wed, 28 Feb 2001
```

`prev_year` is aliased to `last_year`.

15.1.2.2 `prev_month`, `next_month`

In Ruby 1.9 `prev_month` and `next_month` return the date with the same day in the last or next month:

```
d = Date.new(2010, 5, 8) # => Sat, 08 May 2010
d.prev_month             # => Thu, 08 Apr 2010
d.next_month             # => Tue, 08 Jun 2010
```

If such a day does not exist, the last day of the corresponding month is returned:

```
Date.new(2000, 5, 31).prev_month # => Sun, 30 Apr 2000
Date.new(2000, 3, 31).prev_month # => Tue, 29 Feb 2000
Date.new(2000, 5, 31).next_month # => Fri, 30 Jun 2000
Date.new(2000, 1, 31).next_month # => Tue, 29 Feb 2000
```

`prev_month` is aliased to `last_month`.

15.1.2.3 `prev_quarter`, `next_quarter`

Same as `prev_month` and `next_month`. It returns the date with the same day in the previous or next quarter:

```
t = Time.local(2010, 5, 8) # => Sat, 08 May 2010
t.prev_quarter             # => Mon, 08 Feb 2010
t.next_quarter             # => Sun, 08 Aug 2010
```

If such a day does not exist, the last day of the corresponding month is returned:

```
Time.local(2000, 7, 31).prev_quarter  # => Sun, 30 Apr 2000
Time.local(2000, 5, 31).prev_quarter  # => Tue, 29 Feb 2000
Time.local(2000, 10, 31).prev_quarter # => Mon, 30 Oct 2000
Time.local(2000, 11, 31).next_quarter # => Wed, 28 Feb 2001
```

`prev_quarter` is aliased to `last_quarter`.

15.1.2.4 `beginning_of_week`, `end_of_week`

The methods `beginning_of_week` and `end_of_week` return the dates for the beginning and end of the week, respectively. Weeks are assumed to start on Monday, but that can be changed passing an argument, setting thread local `Date.beginning_of_week` or `config.beginning_of_week`.

```
d = Date.new(2010, 5, 8)     # => Sat, 08 May 2010
d.beginning_of_week          # => Mon, 03 May 2010
d.beginning_of_week(:sunday) # => Sun, 02 May 2010
d.end_of_week                # => Sun, 09 May 2010
d.end_of_week(:sunday)       # => Sat, 08 May 2010
```

`beginning_of_week` is aliased to `at_beginning_of_week` and `end_of_week` is aliased to `at_end_of_week`.

15.1.2.5 `monday`, `sunday`

The methods `monday` and `sunday` return the dates for the previous Monday and next Sunday, respectively.

```
d = Date.new(2010, 5, 8)     # => Sat, 08 May 2010
d.monday                     # => Mon, 03 May 2010
```